

Single Agent Systems & Agent Pipelines - Quiz

Instructions:

Answer the following questions. These questions are designed to test your understanding of agent systems, tools, and evaluation.

1. Explain the concept of a stateful directed graph in agent pipelines. How does it differ from a simple linear pipeline?

A stateful directed graph is a way of organizing an agent's workflow where the steps are connected like a map rather than a straight line. Each step is a point in the graph, and the connections decide where things go next. The "stateful" part means the system remembers what happened earlier and can use that information to decide its next move. Because of this, it can branch in different directions, loop back, or skip steps depending on the situation. A linear pipeline, on the other hand, just runs one step after another in a fixed order and forgets everything along the way. So the graph approach is far more flexible when an agent needs to make decisions instead of blindly following a script.

2. Describe the role of nodes and edges in an agent workflow. Give an example of each.

Nodes are the actual pieces of work in the workflow. Each node does one thing, like reading the user's query, running a tool, or writing the final answer. Edges are the arrows between nodes that decide what runs next and how data passes along. In the agent I built, the calculator would be a node, and the connection that sends a math query from the routing step into that calculator would be an edge. Put simply, nodes are the "what happens" and edges are the "what happens next."

3. What is conditional routing in an agent system? Design a simple rule-based routing logic for three different query types.

Conditional routing means the agent looks at the query first and then decides where to send it, instead of treating every query the same way. It's basically the agent picking the right tool for the job. A simple rule-based version could work like this: if the query mentions "calculate" or contains math symbols, send it to the calculator; if it mentions "keywords", send it to the keyword extractor; and if it matches neither, fall back to a general response. This is exactly the logic my agent uses, and the nice thing about rules like these is that they're easy to read and easy to extend when you add a new tool later.

4. Why are cycles (loops) important in agent pipelines? Provide a use case where a retry loop is necessary.

Loops let an agent try something more than once instead of giving up the moment it fails. This matters because real systems hit temporary problems all the time. A good example is calling an external API: if the request fails because the network hiccupped for a second, a retry loop can attempt it again a couple of times before reporting an error. Often the second attempt just works. Without a loop, the agent would quit on the first failure even when the task was perfectly doable, which makes the whole system feel fragile.

5. Explain how a single-agent system can simulate multi-agent behavior internally.

A single agent can act like a small team by splitting its job into separate roles and going through them in order. First it can play the part of an analyzer that figures out what the query is asking. Then it switches to a router that picks the right tool. Finally it acts as a responder that produces the answer. Even though one program is doing all of this, each stage behaves like its own little specialist. This gives you a lot of the benefit of a multi-agent setup without the extra complexity of actually coordinating several agents.

6. What are JSON schema tools? How do they help in structuring tool inputs and outputs?

JSON schema tools are a way of agreeing on a fixed shape for the data that moves between the agent and its tools. The schema spells out which fields are expected, what type each one should be, and what the result should look like. This is helpful because it lets you check that an input is valid before the tool even runs, so bad data gets caught early. It also keeps the outputs consistent, which makes them easy to read and pass along to the next step. In my project, every response comes back as a small JSON object with a "type" and a "result", which is the same idea on a simpler scale.

7. Compare sequential tool calls and parallel tool calls. When would you prefer one over the other?

Sequential calls run one tool at a time, where each step waits for the previous one because it needs that result to continue. Parallel calls fire off several tools at once when none of them depend on each other. You'd go sequential when there's a clear chain, like retrieving a document before summarizing it, since the order genuinely matters. You'd go parallel when the tasks are independent, like fetching weather and stock prices at the same time, because running them together is just faster. The main trade-off is correctness versus speed: sequential keeps dependencies safe, parallel saves time.

8. How would you implement error handling in a tool-using agent? Provide at least two strategies.

The first strategy is wrapping risky operations in try-except blocks so that when something goes wrong, the agent catches it and returns a clear message instead of crashing. My calculator does this, so a query like "10 / 0" comes back as a clean error rather than blowing up the whole run. The second strategy is a retry mechanism, where the agent automatically repeats an operation that failed for a temporary reason before finally giving up. On top of both, logging is worth adding so you have a record of what went wrong, which makes debugging much easier later. Together these keep the agent stable and predictable.

9. What is trajectory evaluation in agent systems? Why is it important beyond just checking final output?

Trajectory evaluation looks at the whole path an agent took to reach its answer, not just the answer itself. It checks the decisions it made, which tools it called, and the intermediate steps in between. This matters because an agent can sometimes land on the right answer for the wrong reasons, or take a messy route that happens to work this time but will fail on the next query. By studying the full trajectory you can spot those weak decisions and fix them. It gives you a much better picture of how the agent actually behaves rather than just whether it got lucky at the end.

10. Define task completion rate and cost metrics. How would you measure and optimize them in a real-world system?

Task completion rate is the share of tasks the agent finishes correctly out of everything it was asked to do. Cost metrics track what each task consumes, things like the number of API calls, processing time, or actual money spent. You measure them by logging results across many runs and watching how often the agent succeeds and how much each success costs. To improve them, you can tighten the routing so queries reach the right tool more often, cut out tool calls that aren't needed, and make the slow parts more efficient. The real goal is a balance: high completion rate without the costs creeping up, since that's what makes a system practical to actually deploy.